

Computer Architecture

CE/CS 321/330

L2

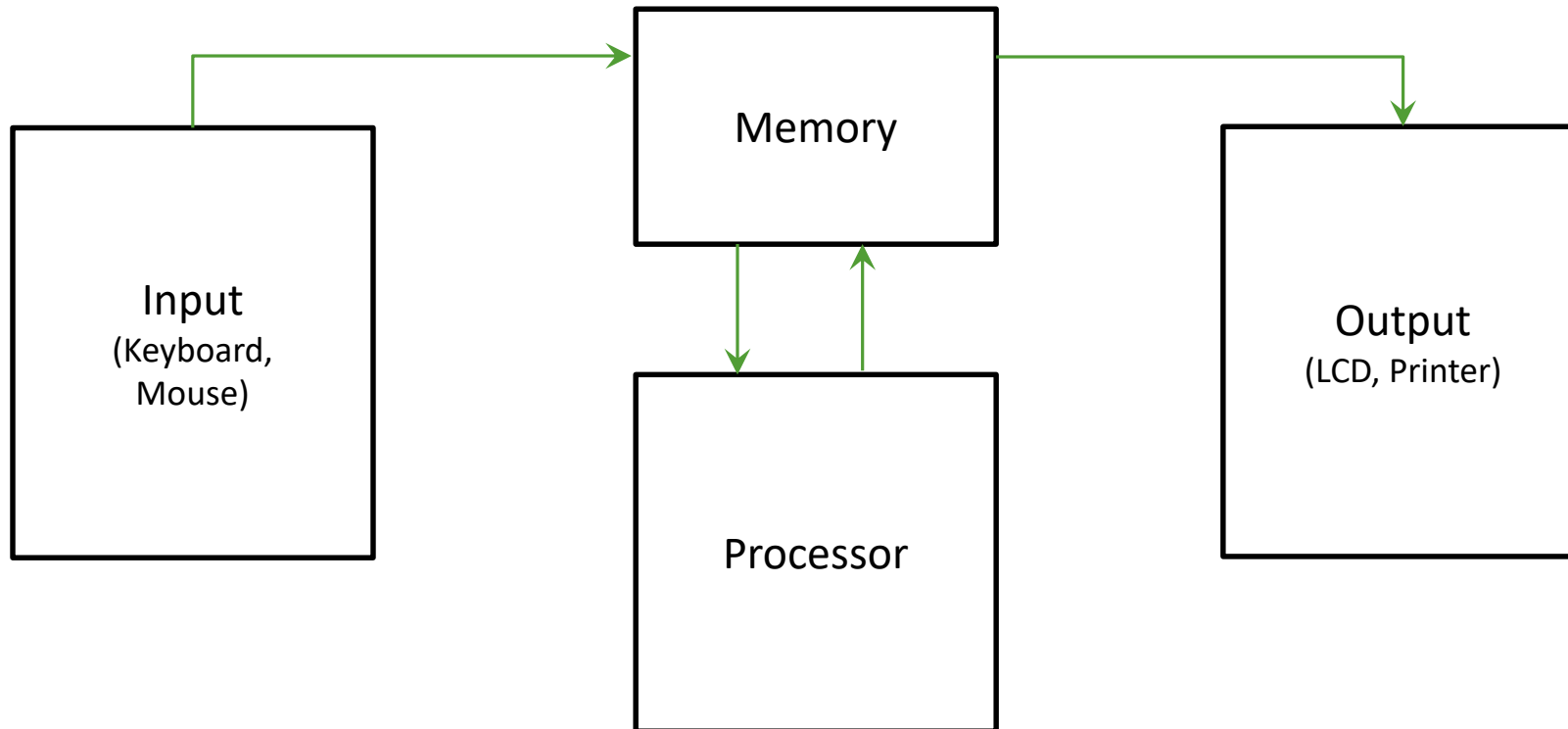
Course Teacher: Dr.-Ing. Shehzad Hasan

Lecture: 24

Outline: Unit 5

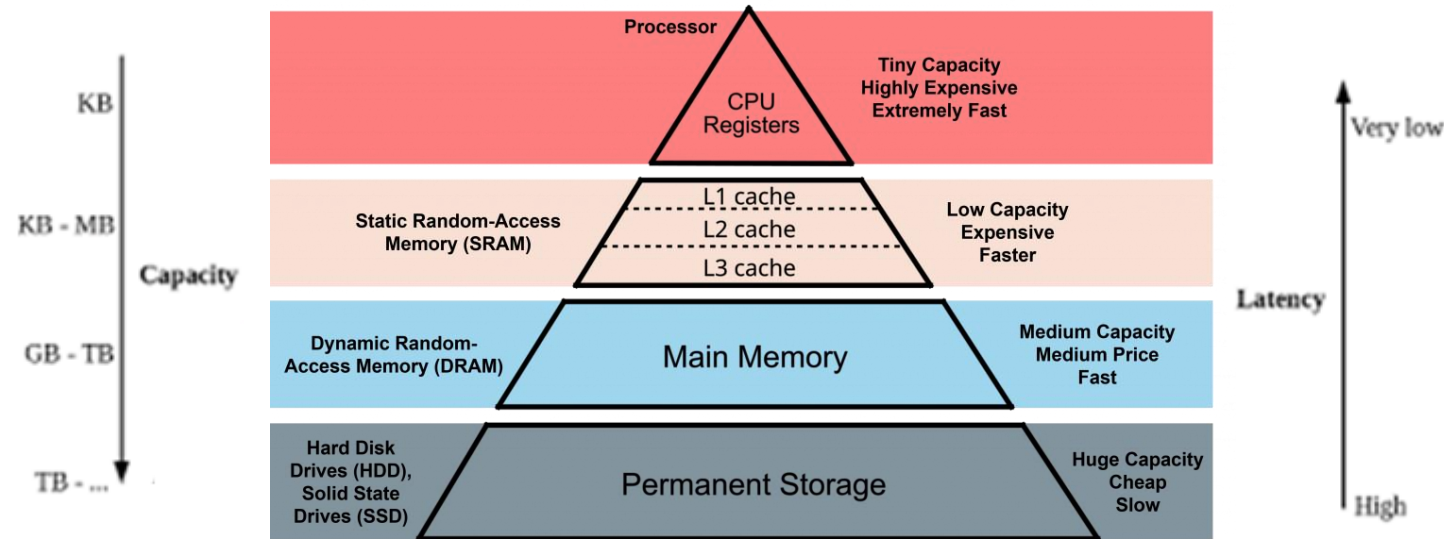
- The Need for Memory Hierarchy
- Cache
 - Block Placement
 - Block Identification
 - Block Replacement
 - Write Strategies
- Measuring Cache Performance
- Multi-Level Cache

Computer Architecture: The Big Picture



Need for Memory Hierarchy

- Ideal Scenario
 - Unlimited amounts of fast memory
- The Problem
 - Fast memory is expensive
 - Cheaper memory is slower to access
- The Solution
 - Memory hierarchy
 - Not just one type of memory but arranging different kinds of storage devices based on their size, cost and access speed
 - Provides the illusion of unlimited fast memory
 - The underlying trick: A program does not access all of its code or data at once with equal probability (Principle of Locality)



Principle of Locality (Locality of Reference)

The programs access a small proportion of their address space at any time

- Spatial Locality
 - If an item is referenced, items whose addresses are close by will tend to be referenced soon
 - Example: sequential instruction access, accessing sequential array elements
- Temporal Locality
 - If an item is referenced, it will tend to be referenced again soon.
 - Example: instructions in a loop, repeated access to the same array element

Spatial and Temporal Localities

- Consider the following program and identify the pattern in which the memory addresses are accessed

```
for (int i = 0; i < SIZE; i++) {  
    arr[i] = arr[i] + arr[i + 1];  
}
```

Let t0 = 0, t1= SIZE, t2 = 0x2000 (arr base address)

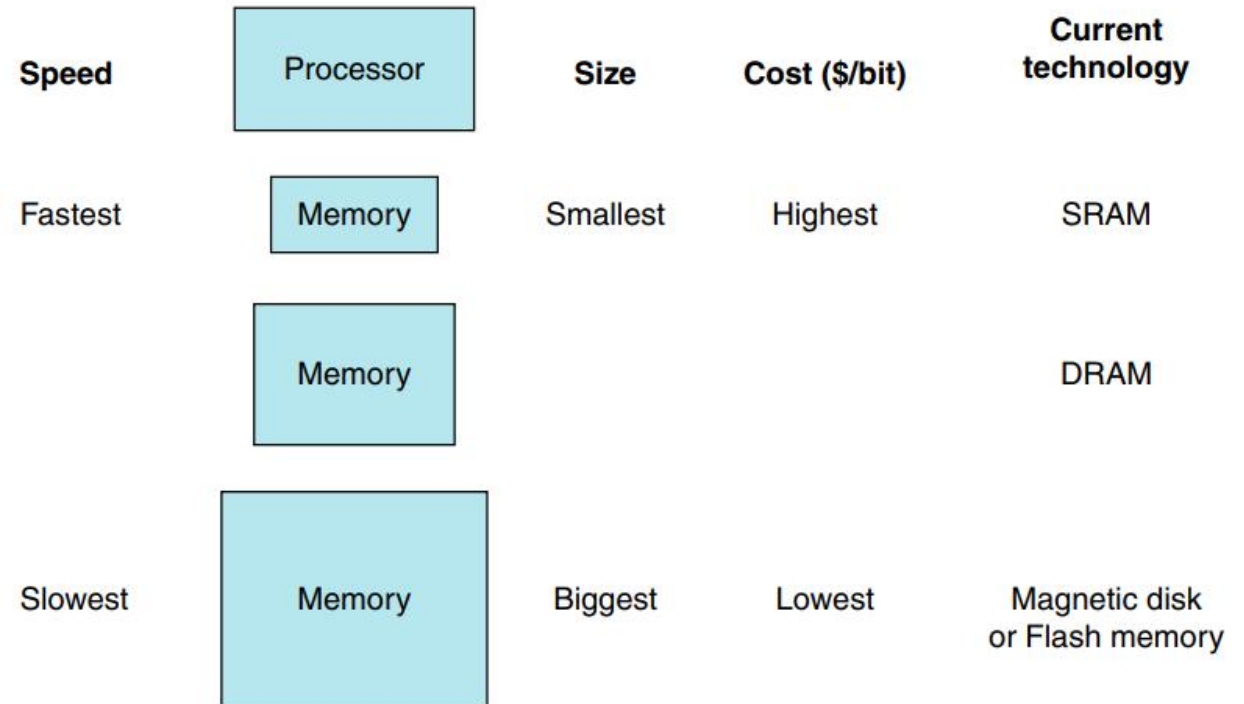
- The instructions are fetched in sequential order but after some cycles the same instructions are repeated.
- The read accesses to data memory are also from nearby locations and we access the same location for writing as well as reading in the next iteration.

0x100	loop:	bge t0, t1, end	# if i >= SIZE, exit loop
0x104		lw t3, 0(t2)	# Load arr[i] into t3
0x108		lw t4, 4(t2)	# Load arr[i+1] into t4
0x10C		add t3, t3, t4	# t3 = arr[i] + arr[i+1]
0x110		sw t3, 0(t2)	# Store result back into arr[i]
0x114		addi t0, t0, 1	# i++
0x118		addi t2, t2, 4	# Move to next element in arr
0x11C		j loop	# repeat SIZE times
0x120	end:		

Taking Advantage of Locality: Memory Hierarchy

- Implement memory system as memory hierarchy

- Consists of multiple levels of memory with different speeds and sizes.
- The user has the illusion of a memory that is as large as the largest level of the hierarchy, but can be accessed as if it were all built from the fastest memory.
- The data stored a level closer to the processor is generally a subset of any level further away, and all the data are stored at the lowest level.



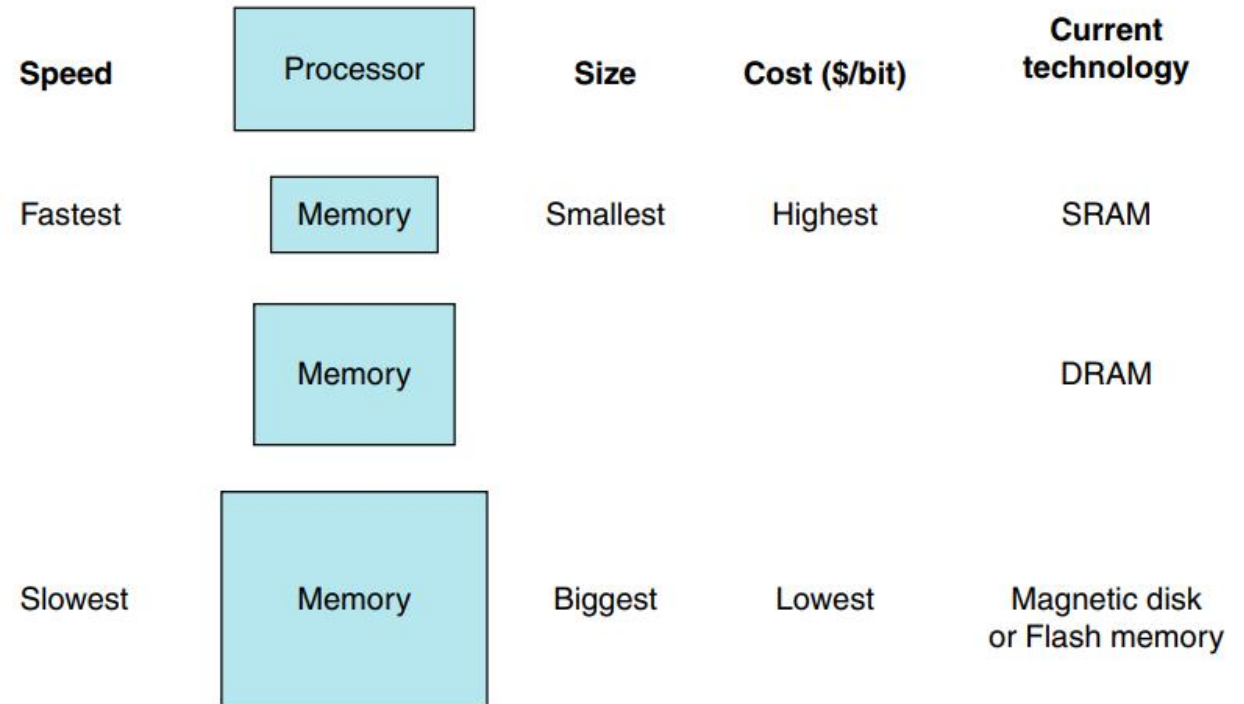
Taking Advantage of Locality: Memory Hierarchy

- Implement memory system as memory hierarchy

- Store everything on disk

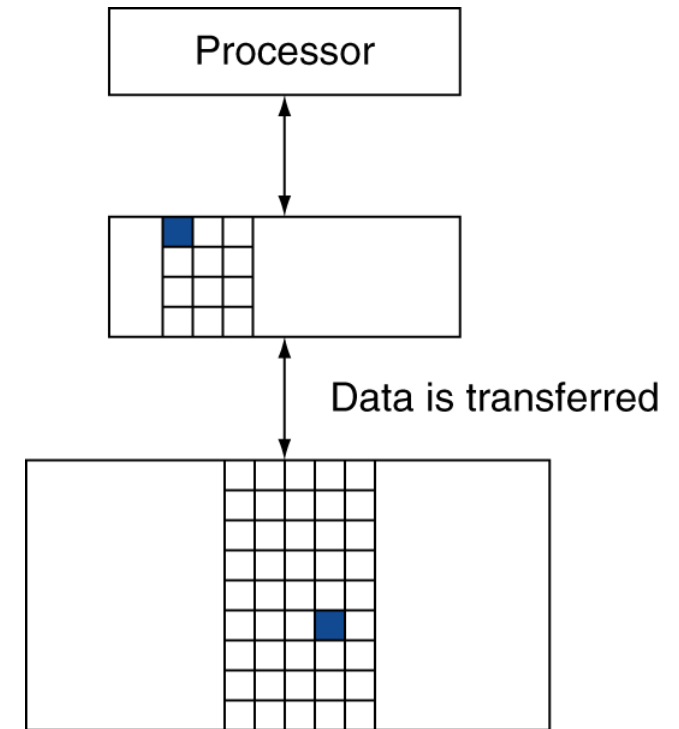
- Copy recently accessed (and nearby) items from disk to smaller DRAM memory (Main memory)

- Copy more recently accessed (and nearby) items from DRAM to smaller SRAM memory (the cache memory attached to CPU)



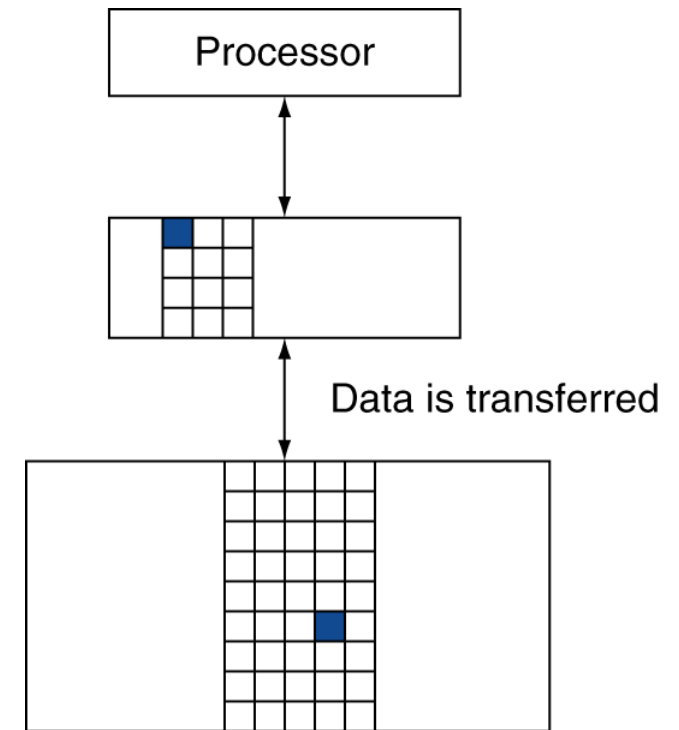
Memory Hierarchy: Terminology

- A memory hierarchy can consist of multiple levels, but data are copied between only two adjacent levels at a time, so we can focus our attention on just two levels.
 - E.g. Cache and Main Memory
- If a word is requested a bigger unit of information containing multiple words is transferred from the lower level (e.g. MM) to the upper level (e.g. cache) to exploit spatial locality. This unit of information is called a **block** or a **line**.
- To keep things simple we will only discuss cache and main memory and refer to unit of transfer as cache lines and main memory blocks. Lines and blocks are the same size.



Memory Hierarchy: Terminology

- Hit
 - If the data requested by the processor appear in some block in the upper level, this is called a hit
- Hit Rate/Ratio
 - The hit rate, or hit ratio, is the fraction of memory accesses found in the upper level; it is often used as a measure of the performance of the memory hierarchy
 - Hit ratio: hits/accesses (probability of finding the data at that level)
- Miss
 - If the data requested by the processor is not found in the upper level, the request is called a miss.
 - The lower level in the hierarchy is then accessed to retrieve the block containing the requested data.
- Miss Rate/Ratio
 - The miss rate is the fraction of memory accesses not found in the upper level.
 - Miss ratio: misses/accesses

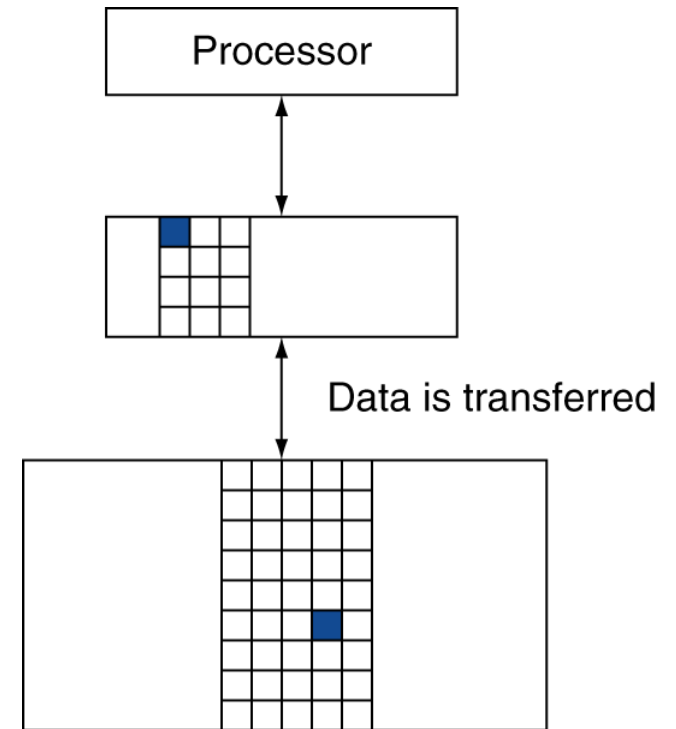


$$\begin{aligned}\text{hit rate} &= 1 - \text{miss rate} \\ \text{miss rate} &= 1 - \text{hit rate}\end{aligned}$$

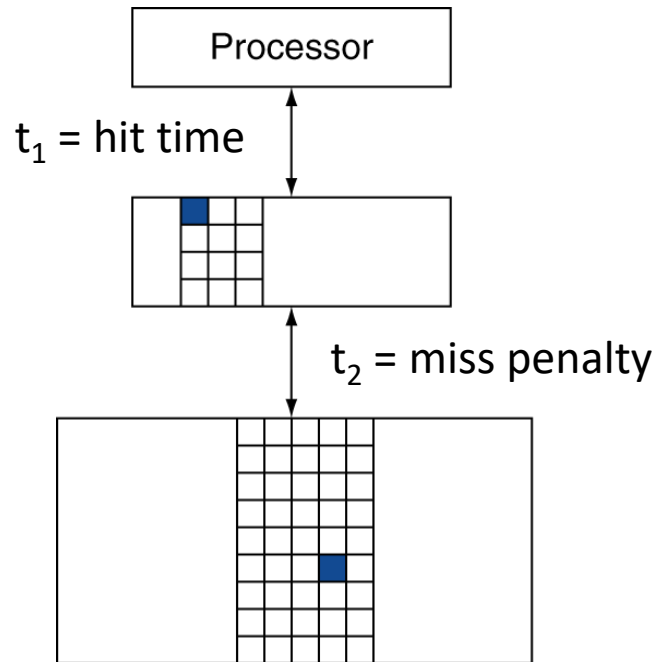
Memory Hierarchy: Terminology

- Hit Time
 - Hit time is the time to access the upper level of the memory hierarchy, which includes the time needed to determine whether the access is a hit or a miss
- Miss Penalty
 - The delay or extra time it takes to retrieve the data from main memory and load it into the cache (block transfer time)

Because the upper level is smaller and built using faster memory parts, the hit time will be much smaller than the time to access the next level in the hierarchy, which is the major component of the miss penalty.



Memory Hierarchy: Terminology



- Average Memory Access Time (AMAT)
 - We want to determine the impact that the memory hierarchy has on the CPU
 - On average, we want to know how long a memory access takes (whether it is from cache or Main memory)

- AMAT = ?

$$\text{AMAT} = h_1 \times t_1 + (1 - h_1) \times h_2 \times (t_1 + t_2)$$

$$\therefore \text{AMAT} = h_1 \times t_1 + (1 - h_1) \times t_1 + (1 - h_1) \times t_2 \quad (\text{For two levels } h_2 = 1)$$

$$\text{AMAT} = t_1 + (1 - h_1) t_2$$

$$\text{AMAT} = t_1 + (1 - h) t_2$$

$$\text{AMAT} = \text{hit time} + \text{miss rate} \times \text{miss penalty}$$

- With a 2-level cache, we can expand our formula:

$$\text{AMAT} = \text{hit time}_1 + \text{miss rate}_1 \times (\text{hit time}_2 + \text{miss rate}_2 \times \text{miss penalty}_2)$$

Memory Hierarchy Basics

- Consider a two-level memory hierarchy with the following attributes:
 - Cache (M_1): 5 ns access time
 - Main memory (M_2): 40 ns access time (block transfer time, miss penalty)

Calculate the average memory access time if 85% of the memory accesses are satisfied (hit) in the cache.

Ans:

$$\text{hit ratio} = ?$$

$$= 0.85$$

$$\text{AMAT} = t_1 + (1 - h) \times t_2$$

$$\text{AMAT} = 5 + (1 - 0.85) \times 40 \text{ ns}$$

$$= 11 \text{ ns}$$

Memory Hierarchy

- Four Memory Hierarchy Questions
 - Q1: Where can a block be placed in the upper level? (block placement)
 - Q2: How is a block found if it is in the upper level? (block identification)
 - Q3: Which block should be replaced on a miss? (block replacement)
 - Q4: What happens on a write? (write strategy)

Memory Hierarchy – Block Placement

- Q1: Where can a block be placed in a cache?

Block placement

- Direct mapped
Only one place
- Fully Associative
Anywhere in the cache
- Set Associative
A block is mapped to a set and then can be placed anywhere within that set

